

Mobile Java objects

A J Herbert, R J Hayton and M Bursell

This paper gives an overview of the design of a mobile Java object infrastructure. It describes why it is helpful to group objects for mobility and how, when clusters represent untrusted pieces of code (for example mobile agents), they must be encapsulated both to control their access to other objects and to control access to them. The difficulties of managing large numbers of mobile objects in an open environment are discussed, together with a solution based on the management of large distributed name spaces. The mobile object system described has been implemented, and is currently being used to support development of mobile agent-based applications within another project. The paper also explains how the infrastructure described meets the requirements of mobile agents better than other approaches.

1. Introduction

Software agents, in particular mobile agents, are currently an active area within both the research community and in industry [1, 2]. However, despite this interest, the commercial exploitation of mobile agent technology has been slow. Often this is diagnosed as being due to the lack of a 'killer application'. However, experience has shown that there are strong opportunities for the use of mobile agents in the automation of networked systems management, intelligent caching of content in large-scale networks and dynamic partitioning of applications between devices, desktop computers and servers.

The more significant reason for the lack of uptake is because mobile agents have generally adopted a different architecture from current approaches to developing distributed applications. This means the transition to agent-based programming may require a programmer to learn a new programming language, lose the support of development tools and be unable to exploit middleware technologies such as CORBA [3] and DCOM [4] to access network services. This creates a chicken and egg situation for agent systems — until they can provide all the services and integration capabilities of current middleware, developers will be reluctant to build large systems using them.

From this observation, it follows that the correct approach to supporting mobile agents is to extend existing distributed programming paradigms by adding autonomy, intelligence and mobility facilities into the virtual machine of an existing distributed object framework. This allows developers to exploit the newer facilities where appropriate, and use standard programming techniques and tools elsewhere.

Java is an ideal language for developing distributed applications. It provides both object and interface abstractions, which gives a useful distinction between an object's interface (i.e. the services it provides) and the implementation of those services. In addition, there are distributed processing capabilities available from Sun's Remote Method Invocation (RMI) [5] and Java implementations of the Object Management Group's CORBA [6, 7].

Sun's RMI is promoted as a pure Java approach. It supports remote invocation, but allows only one interface per object, and insists that objects are constructed from tagged base-classes to indicate if they are to be passed by reference rather than value. This static tagging rules out any possibility of a mobile object offering a service to other objects, and is an unnecessary complication when distributing application components not originally written with distribution in mind. RMI relies heavily on native methods to encode and decode messages from invocations, and therefore the protocols and naming structures used are also statically bound. Stubs to convert local method calls into remote calls are generated by stub-compilers and are tightly coupled to the native methods.

The alternative to RMI is to use a CORBA Object Request Broker (ORB) written in Java. These use the CORBA notions of object and interface, rather than the Java ones; the CORBA interface definition language (IDL) offers a restricted subset of the types supported by Java. Therefore developers have to map between the two models, which is an unnecessary burden. Indeed the principal motivation for using a Java ORB over RMI is the potential to access services written in other languages (e.g. C++), and to exploit CORBA services. Like RMI, ORBs rely on stub-

compilers and consequently are as rigid in their structure as RMI.

Thus, to add mobility to Java, a way has to be found to extend RMI or a CORBA ORB, which is difficult given the closed implementation of these infrastructures. Therefore we chose to build our own open infrastructure. We identified that Java support for language-level introspection and for dynamic code creation makes it possible to design an extensible component-oriented middleware framework [8]. Using reflection, we extend the Java Virtual Machine by enabling selectively transparent remote invocation of any interface. Using Java's introspection facilities we provide a strongly typed framework for binding together infrastructure components. We use Java's support for run-time loading to create stubs on the fly during program execution. This gives an infrastructure we call 'FlexiNet', which is both extremely tailorable and a natural extension of Java's features. When we turned our attention to mobility, we designed a system in which we maintained the FlexiNet view of a 'sea of objects' and extended it by allowing those objects to move between hosts. This movement is transparent to clients of the objects, who simply continue to use Java references to interfaces that have been exported; we call our mobile object system the 'Mobile Object Workbench'.

2. Related work

There are many 'agent' systems available, either commercially or as academic projects. Although some provide high-level abstractions that might be considered as support for intelligent behaviour, the majority of these systems are basically mobile script platforms. They allow scripted agents to move through a distributed network and allow varying degrees of communication between different agents and between agents and hosts or other resources. Examples of these systems include Telescript [9] and Aglets [10].

In general, communication in these systems uses a low-level message model. We believe that the advantages of object-oriented computing, and in particular distributed object computing, are well accepted, and it is equally advantageous to build mobile systems using objects. As, until the advent of the Java programming language, it was difficult to create 'deployable' objects, most mobile code systems relied on scripting technology. It is only recently that the possibility of combining mobility and object orientation has emerged.

Other Java-based mobile object systems work either by abandoning Java's method invocation abstraction and taking the message-passing route, or by utilising RMI. The message-based solutions lose the simplicity of method invocation and strong typing — there is a lack of uniformity between agent interaction and object interaction.

Unfortunately, when designing RMI, Javasoft dictated that each object must be explicitly tagged as a server object or as a data object. Only data objects may be passed by value on a remote invocation. This precludes the possibility of moving server objects and therefore makes RMI inappropriate for use in a mobile object system.

Some systems are based on Java applets. Sadly applets provide class mobility rather than object mobility — an applet cannot carry state with it and can only be downloaded from a central server rather than hop arbitrarily from host to host, which limits the kinds of mobility and communication patterns that can be supported.

Voyager [11] is an exception to the general trend. It is a Java middleware platform that supports mobile services by wrapping them with preprocessor-generated classes. However, objects are moved individually and there is no notion of clustering. Although Voyager is the closest match to our system, we believe we have a more flexible and scalable approach. In FlexiNet, we use introspection, reflection and dynamic linking to modify the behaviour of the Java Virtual Machine; this gives us a more portable solution as it is written completely in Java. It is also more generic as we can use the same mechanisms to add other capabilities such as transactions and security to objects transparently and more dynamically, in that we can decide at run time which capabilities should be bound in for any particular situation. For example, security may not be needed between objects operating within an intranet, but it becomes essential once one of them crosses to the Internet.

3. FlexiNet

The FlexiNet platform is a Java middleware system built to address some of the issues of configurable middleware and application deployment [12]. Its key feature is a component-based 'white-box' approach with strong emphasis placed on generic functions, reflection and introspection. This allows programmers to tailor the platform for a particular application domain or deployment scenario by selecting, adding to or substituting for default infrastructure components. FlexiNet follows closely the computational and engineering components of the ISO reference model for Open Distributed Processing (ODP) [13]. With the addition of mobile objects to FlexiNet, we have implemented the ODP relocation and migration transparencies that most other middleware platforms lack.

3.1 Computational model

We have devised a different computational model in FlexiNet from that adopted in RMI. Invocation parameters which are interface identifiers are passed by reference (i.e. the interface is treated as a service); parameters which are object identifiers are passed by value, i.e. the object is copied. This gives the programmer a simple dynamic way to

determine what semantics apply to the arguments and results of an invocation. Our model allows a mobile object both to be copied (by referencing its object identifier) and to provide a service (by referencing an interface supported by the object).

3.2 Engineering model

In our engineering model, we arrange that interfaces on remote objects are represented locally by ‘proxy objects’. The proxy is transparent to the application programmer. Proxies enforce the typing of the remote interface, and perform remote invocation by utilising ‘binders’. A binder is an object capable of passing a generic invocation between a local proxy object and the remote object it represents. The architecture allows there to be many different binders. Each different binder embodies different application requirements or engineering strategies such as choice of protocols, and system properties such as security or atomicity. Binders may make use of other binders in a recursive way. The most primitive binders are those that wrap a communications protocol, such as remote procedure call. Intermediate binders can add capabilities like access control checking or transactions. This modular architecture encourages the development of a library of simple, ‘pluggable’ infrastructure components and allows application domain-specific binders to be easily inserted or substituted.

An interface can be associated with different binders to permit access from different environments. To manage this flexibility, FlexiNet supports the notion of ‘multiple name spaces’ for interfaces. Names are both strongly typed and structured. Names may be constructed out of other names or arbitrary data and the management of aggregate and indirected names is straightforward. This facilitates the provision of replication and mobility features since names can nominate replicas and can contain hints about how to relocate an object when it has moved.

3.3 Generic communications

Rather than using stubs to convert an invocation directly into a byte array representation, we leverage Java’s run-time typing support to represent the invocation in a generic (but fully typed) form. This is a reflective technique central to the design of FlexiNet. The layers of the FlexiNet communication stack can be viewed as reflexive meta-objects that manipulate the invocation before it is ultimately invoked on the destination object using Java core reflection as illustrated in Fig 1.

This approach allows middleware (or application) components to examine and modify the parameters and semantics of the invocation using the full Java language typing support. Third party meta-objects can be fully general and are fully type-safe. When designing the Mobile Object Workbench, we implemented it as a set of FlexiNet binders, binding protocols, and object location services. In

particular, the mechanism for communication with a mobile object is essentially communication with a static object with two additional reflective layers — a server-side layer, which raises an exception if an object has moved, and a client-side layer, which relocates and rebinds to the object.

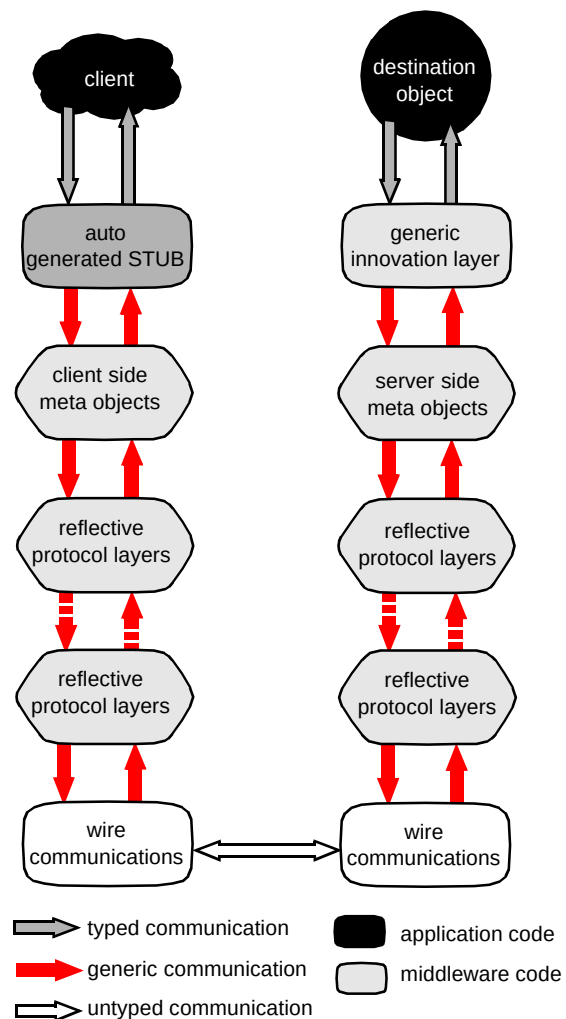


Fig 1 A reflective protocol stack.

4. Requirements for mobile objects

4.1 Relocation

The key feature of a mobile object is that it must be able to move. When we move an object, we effectively copy it to a new location, and then arrange that all references to the old object be replaced with references to the new object. This requires a mechanism for relocation by all previous bindings to the object. If mobile objects were referenced directly using language level pointers, relocation would require changes to the implementation of the Java Virtual Machine, reducing the advantages of Java as a portable language. Instead we arrange that interfaces of mobile objects are referenced indirectly, using FlexiNet stubs as proxies. This level of indirection allows us to relocate referenced interfaces dynamically whenever objects move.

In addition, to avoid the need to track distributed references, we only perform this relocation when a reference to a moved object is first de-referenced, e.g. as part of an invocation. The use of stubs and the rebinding process is transparent to the application programmer, although they may reflect the process, for example, the use of application knowledge to deal with errors.

4.2 Atomicity

At any point in time, a mobile object may be active or passive. An active object is one that has a thread of control currently executing in it, or passing through it. A passive object is one that is not currently being executed, and hence has no threads active in it. When we move an object, we must ensure that the move is atomic. To do this the object must be made passive, i.e. all processing of the object must be suspended. One approach to this would be to pause any threads running in an object, move the object, and then restart the object and the threads at the new location. This, unfortunately, is impractical, as Java does not allow us to determine the complete state of an active thread at an arbitrary point of execution.

Instead, we encapsulate the object and use locks to ensure that no threads are executing the object at the point of movement. This does not prevent the existence of active mobile objects, or those that contain completely internal threads, but it does require a degree of co-operation with the infrastructure, so that they can be 'shut down' prior to movement, and then 'restarted' at the new location. Details are given in section 6.3.

The encapsulation mechanism is integrated with the mechanism for transparent re-binding, so that external threads that have been blocked, pending an object's movement, resume and relocate the newly moved object when it completes its migration.

4.3 Grouping

In the discussion so far, we have been describing the migration of single Java objects. However, there is little purpose in moving a single Java object as they are typically very small, e.g. a linked list will consist of many objects. A more useful unit for mobility is a set of related objects, and we need a mechanism for deciding which parts of a program should move together.

We have taken the notion of a 'cluster' from the reference model for open distributed processing as both a grouping and encapsulating construct. References that pass across a cluster boundary are treated in the way described previously. References entirely within a cluster are ordinary Java language references, as no special action needs to be taken when they are de-referenced.

To a programmer, clusters are a straightforward concept. A special function is used to create the initial object populating a cluster, and after this any new object is

created in the same cluster as its creator. Generally, clusters are completely transparent to the application programmer.

When designing distributed systems, there is always the possibility of host or network failure. In particular, network partition can result in hosts incorrectly assuming that other hosts have failed. When designing a mobile object system, a key decision is the semantics in the worst-case scenario of a network partition during object migration. There are three possibilities. We could allow the possibility of the object existing on both sides of the partition — this was rejected as it introduces an unwanted degree of complexity. The second possibility is to ensure that an object is destroyed if it cannot be uniquely determined on which side of a partition it exists. This is the default semantics chosen in the Mobile Object Workbench. The third possibility is to block indefinitely, until the network is unpartitioned. This can lead to heavy resource usage, but is appropriate for some applications.

4.4 Scalability

There are two issues relating to the scalability of a mobile object system. Firstly, some design choices would require the registration either of all objects or, worse, of all references to all objects. We rejected these approaches as we wish to use the Mobile Object Workbench in an Internet environment, which is both open and has no central administration. The second issue relates to the relocation of interfaces on an object once it has moved. We would like this to be possible, even if the original host has since failed. This issue is discussed in detail in section 8.

5. Requirements for mobile agents

The Mobile Object Workbench was developed as part of an Esprit agent project called 'FollowMe' [14], and one of the other partners in that project is developing an agent system on top of it. As mobile agents are an obvious application of mobile objects, it is worthwhile to review their specific requirements and calibrate our system against them.

5.1 Autonomy

Mobile agents are generally considered to be 'autonomous'. That is to say, it is the agent itself that determines the actions it takes, and in particular controls its movement. In terms of mobile objects and clusters, this gives a requirement for cluster mobility to be initiated only by the cluster itself. The encapsulation mechanism enforces this; only threads within a cluster have access to the objects within it and, by giving one of these objects a handle to the infrastructure which controls mobility, control is effectively hidden from the outside world. There are two exceptions. Firstly, a malicious implementation of the infrastructure can overcome the FlexiNet encapsulation mechanisms — this is a necessary evil of distributed computing — you have to trust the host. The second is that a host may 'legally' destroy an object and reclaim the resources it is using. This is necessary to allow hosts to manage their own resources.

The normal procedure is for a host to inform a cluster that destruction is imminent, in order to allow it to move or shut down cleanly; but a host must always be able to perform a 'forced' shut-down in order to protect itself from malicious or erroneous agents.

5.2 Security

We have found that the approach of implementing a mobile agent as an extension of a mobile object system allowed us to build on established security principles. We identified six basic areas of security concern within the Mobile Object Workbench:

- host integrity — protecting the integrity of a hosting machine, and the data it contains, from possible malicious acts by visiting objects,
- cluster integrity — it should be possible to determine if a cluster has been tampered with, either in transit or by a host at which it was previously located — we may wish to allow hosts to modify some parts of a cluster (e.g. data), but not others (e.g. code),
- cluster confidentiality — a cluster may wish to carry with it information that should not be readable by other clusters, or by (some) of the hosts which it visits,
- cluster authority — a cluster should be able to carry authority with it, e.g. a user's privileges or credit card details, and for this we need both cluster integrity and cluster confidentiality,
- access control — a host should be able to impose different access privileges on different clusters that move to it — clusters and hosts should also be able to enforce access control on exported methods,
- secure communications — clusters and hosts should be able to communicate using confidential and/or authenticated communication — some applications may also require other security communication features, such as non-repudiation.

We believe that unless all of these aspects of security are addressed, a mobile object system will not prove secure enough for real-world applications, and we have therefore

adopted the principle of including security issues from the outset, rather than as an 'add-on', bolted on at a later date. Section 7 discusses our approach to these issues.

5.3 Independence

As clusters representing agents are mutually distrustful pieces of code, it is important that one cluster cannot adversely affect another. In particular, one cluster must not be able to invoke a method on a second cluster, and then destroy the thread performing the call, so as to leave the second cluster in an inconsistent state. Equally, if a cluster crashes or intentionally blocks while servicing a request, the client must be able to recover, and also must neither fail nor block indefinitely.

In order to achieve this degree of strong encapsulation, we decouple all threads that enter or leave a cluster, by spawning and rendezvous, so that the failure of the caller and callee are independent. This thread decoupling is integrated with the binding system and is transparent to the application programmer.

6. The Mobile Object Workbench

6.1 Clusters

Figure 2 shows the relationship between (Java) objects, clusters and mobile objects. A cluster is a Java object containing a set of objects managed together. A mobile object is able to move between places. Places are themselves objects which abstract execution environments, typically with one place per Java Virtual Machine (JVM). Protection, movement, destruction, charging and other management functions are considered in terms of the life cycle of clusters and the interaction between them. It is sometimes useful to consider a cluster and its contents as a virtual process, and the encapsulation and security concerns around clusters encourage this view.

An object is the basic building block out of which applications may be built. Objects may contain references to interfaces on other objects anywhere in the system. Objects may directly create other objects, but only within the same cluster. They may be able to arrange the creation of objects

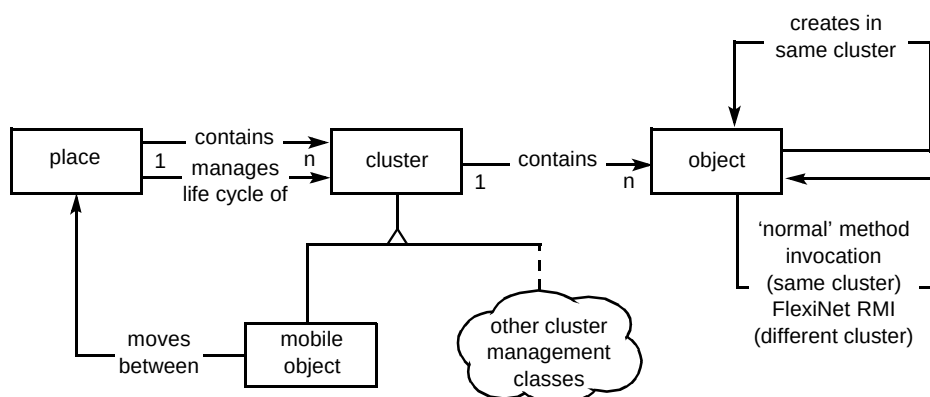


Fig 2 Objects, clusters and mobile objects.

in other clusters via communication with a place. Within a cluster, access to methods/data on objects is determined by standard Java language protection means and takes place using standard Java method invocation. Between clusters, encapsulation is enforced so that objects in one cluster may only access methods on objects in other clusters if these methods form part of an interface passed between the clusters.

As communication between mobile (and non-mobile) objects takes place using application-level exported interfaces, the Mobile Object Workbench application programming interface (API) is entirely concerned with the life cycle of mobile objects, and is analogous to the Applet or Bean API. The key classes and methods are illustrated in Fig 3.

public class Cluster

public synchronised void lock()

public synchronised void unlock()

Change the number of locks held on the object. While a lock is held, new calls made on objects in this cluster from other clusters will block.

public void init()

Called upon object instantiation. A subclass that requires initialisation arguments, or wishes to return an interface to its creator, should provide an alternative `init(...)` method.

public void stop()

A call made by the place if it wishes the cluster to cease processing. The cluster is expected to clean up and then return. When the call returns, the place will destroy the object.

public void restart(Exception e)

Called after the cluster is restarted. A subclass which wishes to take action after a restart should override this method.

public class MobileObject extends Cluster

public lface createObject(Class cls, Object[] args)

Create an object within an empty cluster. Once created, `init(arg0, arg1, ...)` will be called on the new object. The `init` method may return an interface, which is passed to the creator.

public voice syncMove(Place dest)

Request a move to the identified place. The current thread will attempt to perform the move. If successful it will exit. The move will not take place until there are no other threads within the cluster.

public interface Place

public Cluster newCluster()

Create a new cluster at this place. The cluster is created 'empty' and is filled by a call to `Cluster.createObject`.

Fig 3 Mobile Object Workbench API.

6.2 Binding architecture

Communication between clusters takes place by remote method invocation using a 'standard' FlexiNet binder, together with two additional reflexive layers as illustrated in Fig 4. On the client side of an invocation is a 'cluster location' layer. This examines the internal name used to

represent the interface being accessed, and determines the host on which it resides. The procedure adopted is to try the last known location, and only contact the relocation service upon failure.

On the server side of the communication, there is a reflexive 'encapsulation layer'. This processes incoming calls, checks that they refer to clusters that are (still) located on the host, and performs the synchronisation required to ensure that the cluster is not in the process of moving. Part of the encapsulation process is to decouple the calling thread, so that client and server clusters cannot affect each other by 'killing' or otherwise manipulating the thread.

6.3 Orchestrating mobility

Figure 5 illustrates the states in which an instance of a mobile object may be. The object is initially created in state A_1 . This state represents an active object that has one thread in it (the thread that calls the constructor). When active, the object may create other threads. It can also be invoked by objects in other clusters. It will therefore move between active states.

In order to move, a mobile object invokes a `pendMove` or `syncMove` call. Both of these request a move 'as soon as possible', the difference being whether the calling thread returns immediately (`pendMove`) or never returns (`syncMove`). When a move call is invoked, the object enters a pending state. These are identical to active states except that the object will be moved as soon as all executing threads exit (i.e. when it enters state P_0). As a side-effect of executing a `pendMove` or `syncMove`, the cluster becomes locked. When locked, calls from other clusters are blocked until the cluster is unlocked.

When a mobile object enters the state P_0 it will undergo a series of transitions that may result in the creation of a new mobile object at a different place. The original mobile object will then be discarded (it enters state X). If an error occurs during this process and it can be safely inferred that the new object has not been created, this object is then returned to state A_1 . If the move was initialised by a call to `syncMove`, then the error status is returned as an exception. If the move was initiated by a call to `pendMove`, the restart method is called, and the exception is passed as a parameter.

The newly moved cluster is an exact replica of the original, and, in addition, all references to interfaces exported by the original cluster are remapped to the new one (effectively the original cluster has moved). It is started in state A_1 by a call to `restart`. The new cluster (or original after failure) will have the same lock status as the original — apart from the lock automatically taken when `pendMove` or `syncMove` was called, which is released. If a cluster wishes to restart in a locked state, then it may obtain an additional lock prior to calling `pendMove` or `syncMove`. This allows newly moved clusters to perform start-up cleanly, before allowing external access.

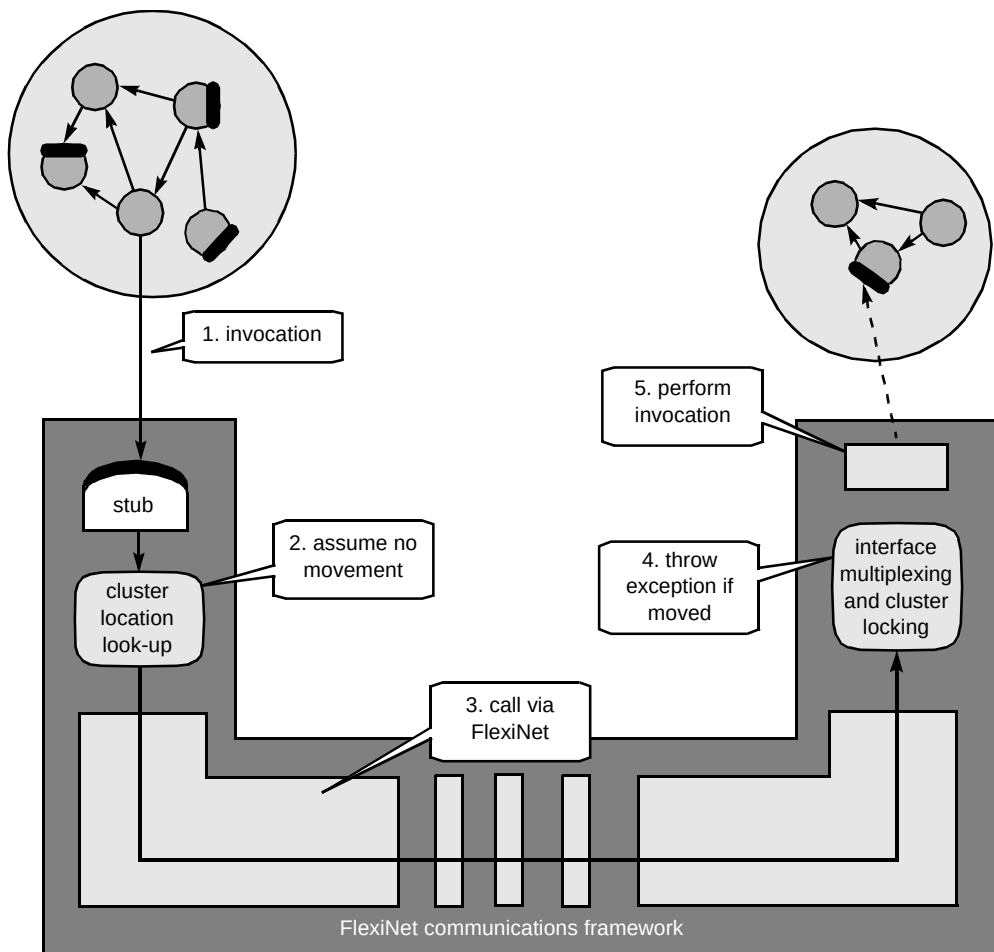


Fig 4 Implementation of inter-cluster calls.

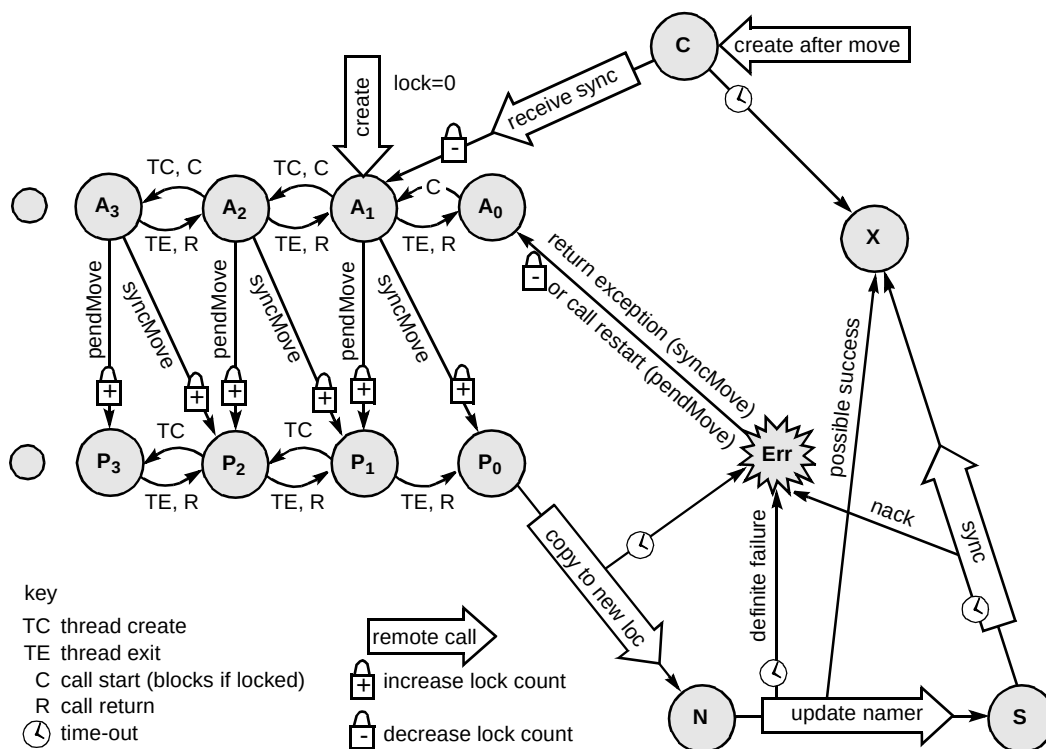


Fig 5 State transitions of a mobile object.

Copying, rather than moving, a cluster follows exactly the same procedure as `syncMove`. The copy operation blocks until there are no other threads and the new cluster has been created, or a failure is detected. After successful synchronisation, or failure, the original object enters state A_1 and the copy operation terminates. The newly created copy commences operation with a call to `copied` in state A_1 .

6.4 Method invocation

When an object in one cluster attempts to invoke a method on an object in another cluster, it must block if the callee cluster is in the process of moving. The state diagram in Fig 6 indicates the process followed by the infrastructure. It should be noted that the callee is able to interrupt a thread making a call, but that this will not affect the caller. This prevents the caller from blocking the callee's progress.

7. Security

Within a mobile object context, issues of trust take on a different slant to non-mobile systems. In both mobile and non-mobile systems, questions of how much trust is placed in an object must be based on the provenance of that object — where it originated, and its history. In a non-mobile object-oriented system, such as the base Java implementation, objects are typically instantiated from class

files, having no other state. In Java, these classes may be signed, and a JVM may assign policies to their instantiations based on this signing. In a mobile object context, an object arriving from a remote host has a history that is not captured by the signature on the class file. Class signatures alone do not provide sufficient information on which to assign security policy.

For this reason, we have designed and implemented a security manager that extends Java's model by allowing policies to be assigned to instances of objects, rather than just their class. This has been possible because of the strong thread encapsulation we have employed within the Mobile Object Workbench, which gives each cluster its own thread group. As Java allows checking of the thread performing a particular operation, we may determine the cluster from which an invocation originated, and hence enforce the appropriate policies.

This security policy allows hosts to restrict operations allowed by particular clusters, thereby protecting their own integrity. It also provides a good base from which to extend cluster-to-cluster access restrictions.

Cluster integrity and confidentiality are enforced by encrypting and/or signing certain objects within a cluster. This prevents a host without sufficient access privileges

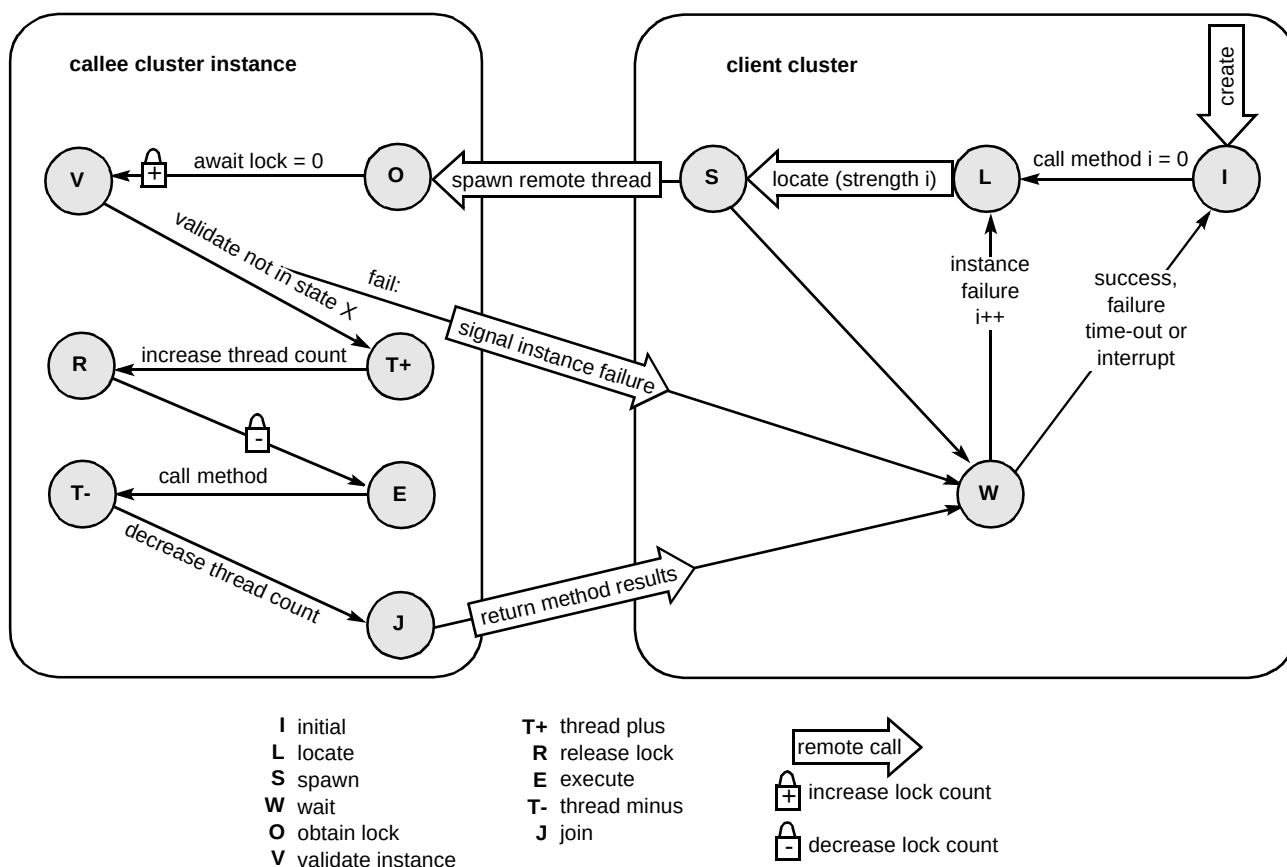


Fig 6 State transitions for inter-cluster calls.

from examining a cluster's state. It also allows one host to detect when a cluster has been modified by a previously visited host. In addition to this, we must ensure that clusters are not dissected — otherwise a malicious host could 'steal' parts of the cluster that represented encrypted passwords and use them to build its own clusters. To do this, we require a mechanism for specifying, and validating, integrity statements. For example, we may annotate a cluster's definition to indicate that a specified field may only be modified by selected hosts. We may then use digital signature techniques to ensure that whenever the field is modified it obtains a signature from the current host, and when other hosts attempt to read this field we can throw an exception if the signature is incorrect.

We are currently developing a system to allow integrity policy statements to be specified. Once specified, the use of secure fields or objects can be made almost transparent to the programmer. All that is required is that they use accessor functions to access the protected fields.

Cluster authority can be implemented using cluster integrity and confidentiality. Together these allow a cluster to carry with it a password or other secret information, without the concern that this secret can be read at any host which is visited. Clearly, once the secret is revealed to a host, there is nothing that can be done to prevent the host from misusing it. For this reason our model ensures that the mobile object moves into a secure environment before revealing a secret. Figure 7 gives an example; a cluster may move between several hosts before eventually arriving at a 'Bank' host. At this host, it may reveal a password to allow it access to a bank account. However, as the Bank host

already knew the password, revealing it has not given the bank any additional privileges, and the security of the password has not been weakened.

Access control and secure communications may be implemented using standard techniques. We use FlexiNet's reflective binding system to allow a cluster to receive notification of an invocation immediately prior to its execution, so that it may implement its own security policy, and throw an access control exception if appropriate. Secure communication between places may take place using a FlexiNet binder that supports the SSL (secure sockets layer) protocol [15]. Secure communication between mobile clusters may also use SSL, but it requires that clusters reveal the information used to prove their identity to the host from which they are communicating. This is reasonable in some circumstances, but should be used with caution.

8. Name relocation service

When a call is made on a mobile cluster, the encapsulation layer at the called host will determine whether the cluster (still) exists at this host. If it does not, the host will raise an exception which is passed back to the callee and caught in the callee's locate layer. This then contacts the name relocation service to determine the new location of the object. The relocation service is a federation of a number of directories. Each directory contains a mapping from old to new cluster addresses. Our naming service was developed with five key properties:

- we control which entities are able to update the directories, i.e. only hosts from which a cluster is moving may update the record for the cluster — this is

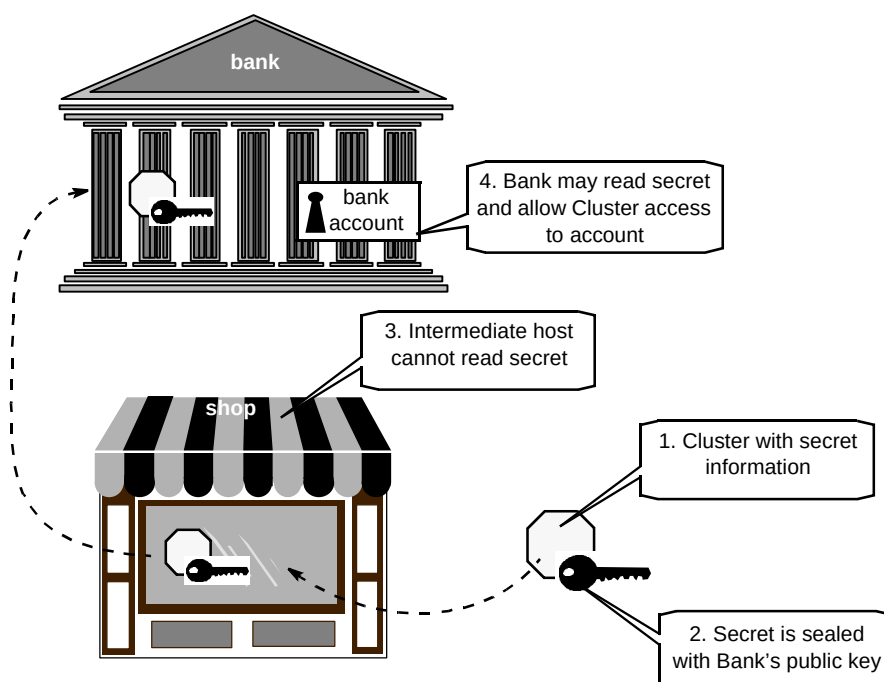


Fig 7 Clusters with secrets.

possible as cluster names (transparently to the applications programmer) contain information about their current network host, thus preventing fraudulent changing of naming records by 'spoof' hosts or clusters,

- we provide a hierarchy of directories, for scale and robustness — this means that an instance of the relocation service may decide to copy the naming record for a cluster up the hierarchy to increase its stability, or to reduce the load placed upon it,
- we allow naming records to be moved between directories so that an optimal directory location can be chosen for the record, e.g. following the movement of a cluster around the network,
- we allow caching for performance — a naming record can be kept at a previous host, as well as being passed up the hierarchy, to reduce look-up time,
- we arrange that a client can locate the appropriate directory for a cluster rapidly, without having to search other directories.

9. Implementation status

The Mobile Object Workbench version 2.0 is currently in use by the members of the Esprit FollowMe project [14]. The current implementation supports clusters, mobility and transparent communications as described in this paper. Additional features developed beyond those described in this paper include:

- a federated network class loader, to allow support for mobile objects that have different views on the Java class name space — using this class loader, a place can support two clusters that have different interpretations on the mapping between the class named 'A', and the code implementing this class (this feature is important in an Internet environment where there is no global consensus on class names — and to support evolution of classes),
- a security manager, which can enforce different security policies based on the identity of the cluster invoking a call,
- the current implementation of the relocation service does not support the migration of directory entries — we intend to rectify this in the near future,
- support for secure bindings using SSL,
- support for CORBA interoperability using the Internet inter-orb protocol (IIOP),
- a transaction system conforming to the Enterprise Java Bean architecture [16].

10. Conclusions

The Mobile Object Workbench was designed primarily to add mobility transparency to the distribution transparencies provided by FlexiNet. In this paper we have shown that in order to do this we must add clustering and re-binding mechanisms. If mobile objects are to be used to support autonomous agent systems, the security requirements then lead to the need for encapsulation mechanisms so that hosts and agents may communicate and co-operate without the need for complete trust. We have approached the design of the Mobile Object Workbench as a distributed system problem, as it has all the traditional issues related to distributed systems — scale, robustness, independent failure modes, distrust, decentralised administration, multiple name spaces and diverging code bases. This approach has led us to design an architecture, and implementation, that can evolve to meet future needs, and we believe this gives it clear advantages over the *ad hoc* approaches of existing mobile agent systems. In addition, we have designed the system as a natural extension of the Java language. This makes it straightforward to use, and allows programmers of mobile objects to use the full language facilities.

Acknowledgements

The work reported in this paper formed part of the Advanced Networked Systems Architecture (ANSA) collaborative research programme on distributed computing. BT has been a member of ANSA from its inception in 1985 through to its completion in 1998 with seconded BT staff working in the ANSA laboratory in Cambridge. BT researchers have used ANSA results including the ANSA architecture, Object Request Broker/Distributed Processing Environment prototypes and Web application server prototypes in BT research projects and standards activities. Since 1997, the focus of the ANSA research has been on the design of middleware for Java, with a particular strand directed towards support for mobile agents. The authors are with Citrix Systems (Cambridge) Ltd, formerly APM Ltd, who host and co-ordinate ANSA.

References

- 1 'Mobile Agents 97', First International Workshop on Mobile Agents 97, Berlin (April 1997) — <http://www.informatik.uni-stuttgart.de/ipvr/vs/ws/ma97/ma97.html>
- 2 'Agent Product and Research Activities', Agent Society (1998) — <http://www.agent.org/pub/activity.html>
- 3 Object Management Group (OMG): 'CORBA/IIOP 2.1 Specification', (1997) — <http://www.omg.org/corba/corbiio.htm>
- 4 Brown N and Kindel C: 'Distributed component object model protocol — DCOM/1.0 — network working group INTERNET-DRAFT', Microsoft (1998) — <http://www.microsoft.com/oledev/olecom/draft-brown-dcom-v1-spec-02.txt>
- 5 Sun Microsystems: 'Java remote method invocation (RMI) Specification', (1996) — <http://www.sun.com/products/jdk/1.1/docs/guide/rmi/>

- 6 Iona: 'OrbixWeb — A Java ORB from IONA Technologies', (1998) — <http://www.orbix.com/products/internet/orbixweb/>
- 7 Visigenics: 'VisiBroker — A Java ORB from VisiGenics', (1998) — <http://www.visigenic.com/>
- 8 Hayton R J and Herbert A J: 'A flexible component oriented middleware system', SIGOPS European Workshop'98 (1998).
- 9 'General Magic Telescript Language Reference', General Magic Inc (1995)
- 10 Lange D B and Change D T: 'Aglets Workbench', (1996) — <http://www.ibm.co.jp/trl/aglets>
- 11 ObjectSpace: 'Voyager Core Technology', (1997) — <http://www.objectspace.com/Voyager/>
- 12 APM: 'FlexiNet — Automating application deployment and Evolution', (1996) — <http://www.ansa.co.uk/Research/FlexiNet.htm>
- 13 ODP: 'ODP Reference Model', (1995) — http://www.dstc.edu.au/AU/research_news/odp/ref_model/
- 14 FAST: 'FollowMe project overview', (1997) — <http://hyperwav.fast.de/generalprojectinformation>
- 15 Netscape Inc: 'The SSL Protocol', (1996) — <http://home.netscape.com/newsref/std/SSL.html>
- 16 Sun Microsystems: 'Enterprise JavaBeans™', (1997) — <http://www.javasoft.com:80/products/ejb/docs.html>



Andrew Herbert is Director of Advanced Technology for Citrix Systems Inc. He led the ANSA industry-sponsored programme of research and advanced development into the use of distributed systems technology from its start in 1985 through to its completion in 1998 with the work reported in this paper.

His interests are in the application of object technology to the deployment and management of distributed systems.



Richard Hayton graduated from Cambridge University in 1991 and joined the department as a research associate investigating LAN multimedia. He was awarded a PhD in 1996 for work on an Open Architecture for Secure Interworking Services (OASIS). He joined Citrix Systems (Cambridge) in October 1996 and has since been leading work on a flexible Java middleware platform (FlexiNet). His current research interests include support for complex trust relationships in distributed systems.



Michael Bursell graduated from Cambridge University in 1994 and spent two years with Cambridge University Press engaged in electronic publishing projects and Internet development. Since joining Citrix Systems (Cambridge) in 1996 he has worked on Java middleware, database integration and FollowMe. Much of his work has focused on issues of mobile code, from Java Applets and RMI to push technologies and mobile agents, and he has a particular interest in issues of security around mobile object systems.